Date:-29/10/2026

Week-4

Cuckoo Search (CS):

Pseudocode

```
BEGIN
  // STEP 1: Initialize Parameters
  Set n = Number of nests
  Set pa = Discovery probability
  Set MaxIter = Maximum iterations
  Set alpha = Step size
  Set lambda = Levy distribution parameter
  Set Lower = Lower search boundary
  Set Upper = Upper search boundary

  // STEP 2: Initialize Population
  FOR i = 1 TO n DO
    X[i] = Lower + rand(0,1) * (Upper - Lower)
    Fitness[i] = f(X[i])
  END FOR

  // STEP 3: Find Best Nest
  BestNest = Nest with minimum fitness
  BestFitness = f(BestNest)

  // STEP 4: Repeat Until Maximum Iterations
  FOR t = 1 TO MaxIter DO
```

```
// STEP 5: Generate New Solutions using Levy Flight
FOR i = 1 TO n DO
   LevyStep = Levy(lambda)
   Xnew = X[i] + alpha * LevyStep

   // Keep solution within boundaries
   Xnew = ApplyBounds(Xnew, Lower, Upper)
   NewFitness = f(Xnew)

   // STEP 6: Select a Random Nest
   Randomly select nest j
   IF NewFitness < Fitness[j] THEN
      X[j] = Xnew
      Fitness[j] = NewFitness
   END IF
END FOR

// STEP 7: Abandon Worst Nests
NumberAbandoned = ROUND(pa * n)
Identify the worst NumberAbandoned nests

FOR each abandoned nest j DO
   Generate a new random solution
   X[j] = Lower + rand(0,1) * (Upper - Lower)
   Fitness[j] = f(X[j])
END FOR

// STEP 8: Update Best Solution
Find nest with minimum fitness
```

```
    IF BestFitness is greater than minimum fitness THEN
      BestNest = Current best nest
      BestFitness = Minimum fitness
    END IF
    PRINT iteration and BestFitness
  END FOR

  // STEP 9: Display Final Result
  PRINT BestNest
  PRINT BestFitness

END
```

Python Program: Cuckoo Search for Clustering Mixed Data (CCS-K-Prototypes)

```
import random
import math
import matplotlib.pyplot as plt

# STEP 1: DATASET

data = [
  ["male",   18, 175, "writing"],
  ["female", 24, 165, "music"],
  ["female", 23, 175, "tennis"],
  ["male",   45, 185, "football"],
  ["male",   35, 195, "basketball"],
  ["male",   26, 180, "tennis"],
  ["female", 22, 172, "music"]  ]
```

```
# Numeric and categorical attributes

numeric_cols = [1, 2]
categorical_cols = [0, 3]

n = len(data)
k = 2

# STEP 2: INITIALIZE PARAMETERS
N = 15               # Number of nests
pa = 0.20            # Abandon probability
maxGen = 2500        # Maximum generations
beta = 1.5           # Levy distribution parameter
alpha = 0.01         # Step size coefficient
mu = 1.0             # Categorical attribute weight

random.seed(42)

# Calculate numeric boundaries
lower = {}
upper = {}

for col in numeric_cols:
    lower[col] = min(row[col] for row in data)
    upper[col] = max(row[col] for row in data)

# STEP 3: INITIALIZE NESTS
def initialize_nest():

    # Select k random data objects as cluster centres
```

```
    return [
        data[i].copy()
        for i in random.sample(range(n), k)
    ]

nests = [initialize_nest() for _ in range(N)]

# STEP 4: DISTANCE CALCULATION
def mixed_distance(row, centre):
    total = 0

    # Numeric distance (normalized squared difference)
    for col in numeric_cols:
        value = ((float(row[col]) - float(centre[col])) / (upper[col] - lower[col]) ) ** 2
        total += value

    # Categorical distance
    for col in categorical_cols:

        if row[col] != centre[col]:
            total += mu
    return total

# STEP 5: FITNESS FUNCTION
def fitness(nest):
    total_cost = 0

    for row in data:
```

```
        # Find nearest cluster centre
        distances = [mixed_distance(row, centre)
            for centre in nest
        ]

        total_cost += min(distances)
    return total_cost

# STEP 6: LEVY FLIGHT
# Mantegna's Algorithm
def levy_flight():
    sigma_u = (math.gamma(1 + beta) * math.sin(math.pi * beta / 2)  /( math.gamma((1 + beta) / 2)
            * 2 ** ((beta - 1) / 2)
        )
    ) ** (1 / beta)

    u = random.gauss(0, sigma_u)
    v = random.gauss(0, 1)

    step = u / (abs(v) ** (1 / beta) + 1e-10)
    return step

# STEP 7: APPLY BOUNDARIES
def apply_bounds(value, col):
    return max(
        lower[col],
        min(upper[col], value)
    )
```

```
# STEP 8: LOCAL SEARCH
# Equation (11) of the paper
def get_next_solution(current, best):

    new_nest = []

    for cluster in range(k)
        new_centre = []
        for col in range(len(data[0])):
            if col in numeric_cols:
                step = levy_flight()
                old_value = float(current[cluster][col])
                best_value = float(best[cluster][col])
                new_value = (
                    old_value
                    + alpha * step
                    * abs(best_value - old_value)
                )

                new_value = apply_bounds(new_value, col)

                new_centre.append(new_value)

            else:
                # Randomly select current or best category
                new_value = random.choice([
                    current[cluster][col],
                    best[cluster][col]
                ])
                new_centre.append(new_value)
```

```
        new_nest.append(new_centre)

    return new_nest

# STEP 9: GLOBAL SEARCH
# Equation (17) of the paper
def get_new_solution(abandoned, nest_f, nest_g):
    new_nest = []
    for cluster in range(k):
        new_centre = []

        for col in range(len(data[0])):
            if col in numeric_cols:
                epsilon = random.random()
                old_value = float(abandoned[cluster][col])
                value_f = float(nest_f[cluster][col])
                value_g = float(nest_g[cluster][col])

                new_value = (
                    old_value
                    + epsilon * abs(value_f - value_g)
                )
                new_value = apply_bounds(new_value, col)
                new_centre.append(new_value)

            else:
                new_value = random.choice([
                    abandoned[cluster][col],
                    nest_f[cluster][col],
                    nest_g[cluster][col]
```

```
            ])

            new_centre.append(new_value)

        new_nest.append(new_centre)

    return new_nest

# STEP 10: MAIN CUCKOO SEARCH ALGORITHM
# Evaluate initial nests

fitness_values = [fitness(nest) for nest in nests]

best_index = fitness_values.index(min(fitness_values))

best_nest = [
    centre.copy() for centre in nests[best_index]
]

best_fitness = fitness_values[best_index]

fitness_history = []

for generation in range(maxGen):
    # LOCAL SEARCH
    current_index = random.randrange(N)

    new_nest = get_next_solution(
        nests[current_index],
        best_nest
```

```
)
new_fitness = fitness(new_nest)

# Update best nest

if new_fitness < best_fitness:

    nests[best_index] = [
        centre.copy() for centre in new_nest
    ]

    best_nest = [centre.copy() for centre in new_nest]
    best_fitness = new_fitness
    best_index = nests.index(nests[best_index])

# GLOBAL SEARCH / ABANDONMENT

if random.random() < pa:
    # Find worst nest
    fitness_values = [fitness(nest) for nest in nests]
    worst_index = fitness_values.index(
        max(fitness_values)
    )

    # Select two random nests

    indices = [
        i for i in range(N)
        if i != worst_index
    ]
```

```
    f, g = random.sample(indices, 2)
    # Generate new nest
    new_global_nest = get_new_solution(
        nests[worst_index],
        nests[f],
        nests[g]
    )

    new_global_fitness = fitness(new_global_nest)

    # Replace worst nest if better
    if new_global_fitness < fitness_values[worst_index]:
        nests[worst_index] = new_global_nest
        if new_global_fitness < best_fitness:
            best_nest = [
                centre.copy()
                for centre in new_global_nest
            ]
            best_fitness = new_global_fitness
            best_index = worst_index

# Store best fitness
fitness_history.append(best_fitness)

if (generation + 1) % 500 == 0:

    print(
        "Generation:", generation + 1,
        "Best Fitness:", round(best_fitness, 5)
    )
```

```
# STEP 11: FINAL CLUSTER ASSIGNMENT
clusters = [[] for _ in range(k)]
for i, row in enumerate(data):
    distances = [
        mixed_distance(row, centre)
        for centre in best_nest
    ]

    cluster_number = distances.index(min(distances))
    clusters[cluster_number].append(i + 1)

# STEP 12: DISPLAY RESULTS
print("\n========== CUCKOO SEARCH RESULTS ==========")
print("\nBest Cluster Centres:")

for i, centre in enumerate(best_nest):
    print("Cluster", i + 1, ":", centre)
print("\nFinal Clustering:")

for i, cluster in enumerate(clusters):
    print("Cluster", i + 1, ":", cluster)
print("\nBest Fitness:", round(best_fitness, 5))

print("==========================================")
# STEP 13: GRAPH 1 - CONVERGENCE

plt.figure(figsize=(9, 5))
plt.plot(fitness_history, linewidth=2)
plt.title("Cuckoo Search - Convergence Graph")
```

```
plt.xlabel("Number of Generations")
plt.ylabel("Best Fitness Value")
plt.grid(True)
plt.tight_layout()

# STEP 14: GRAPH 2 - CLUSTERING
plt.figure(figsize=(9, 5))
for i, cluster in enumerate(clusters):
    ages = [data[j - 1][1] for j in cluster]
    heights = [data[j - 1][2] for j in cluster]

    plt.scatter(ages, heights, s=100, label="Cluster " + str(i + 1))

for i, row in enumerate(data):

    plt.annotate(
        "ID " + str(i + 1),
        (row[1], row[2]),
        xytext=(5, 5),
        textcoords="offset points"
    )
plt.title("Cuckoo Search - Data Clustering")
plt.xlabel("Age")
plt.ylabel("Height")
plt.legend()
plt.grid(True)
plt.tight_layout()

plt.show()
```

⇨ OUTPUT

```
Generation: 500 Best Fitness: 5.07096
Generation: 1000 Best Fitness: 5.07096
Generation: 1500 Best Fitness: 5.07096
Generation: 2000 Best Fitness: 5.07096
Generation: 2500 Best Fitness: 5.07096

========== CUCKOO SEARCH RESULTS ==========

Best Cluster Centres:
Cluster 1 : ['male', 26, 180, 'tennis']
Cluster 2 : ['female', 22, 172, 'music']

Final Clustering:
Cluster 1 : [1, 4, 5, 6]
Cluster 2 : [2, 3, 7]

Best Fitness: 5.07096
==========================================
```



